

LAPORAN TUGAS BESAR
IF2224 TEORI BAHASA FORMAL DAN OTOMATA

MILESTONE 3



Disusun oleh:

Gabriella Botimada Lubis - 13524006

Stefani Angeline Oroh - 13524064

Reva Natania Sitohang - 13524098

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG

2026

DAFTAR ISI

DAFTAR ISI.....	2
DAFTAR TABEL.....	4
DAFTAR GAMBAR.....	5
BAB I	
DASAR TEORI.....	6
1.1 Semantic Analysis.....	6
1.2 Abstract Syntax Tree (AST).....	6
1.2.1 Implementasi Token.....	7
1.3 Symbol Table.....	8
1.3.1 tab - Identifier Table.....	8
1.3.2 btab - Identifier Table.....	8
1.3.3 atab - Identifier Table.....	9
1.4 Predefined Identifiers.....	9
1.5 Type Compatibility.....	10
1.6 Scope dan Lexical Level.....	10
BAB II	
PERANCANGAN DAN IMPLEMENTASI.....	11
2.1 Arsitektur Program.....	12
2.1.1 Alur Eksekusi Program.....	13
2.1.2 Dependensi Antar Modul.....	14
2.1.3 Dua Fase Semantic Analyser.....	14
2.1.4 Justifikasi Pemilihan Implementasi.....	15
2.2 Implementasi Kode.....	16
2.2.1 Symbol Table (symbol_table.hpp / .cpp).....	16
2.2.2 Definisi AST Node (ast.hpp).....	18
2.2.3 buildAST : Syntax-Directed Translation (semantic.cpp).....	19
2.2.4 visitProgram: Dekorasi AST (semantic.cpp).....	21
2.2.5 visitVarDecl: Registrasi Variabel (semantic.cpp).....	22
2.2.6 visitProcDecl / visitFuncDecl: Registrasi Subprogram (semantic.cpp).....	23
2.2.7 visitTypeDecl: Registrasi Tipe Record (semantic.cpp).....	24
2.2.8 Type Checking: visitBinOp dan resultTypeOfBinOp (semantic.cpp).....	25
2.2.9 visitAssign: Pengecekan Assignment Compatibility (semantic.cpp).....	26
BAB III	
PENGUJIAN.....	28
3.1 Test Case 1.....	28
3.2 Test Case 2.....	29
3.3 Test Case 3.....	32
3.4 Test Case 4.....	34

3.5 Test Case 5.....	37
3.6 Test Case 6.....	39
3.7 Test Case 7.....	42
3.8 Test Case 8.....	44
3.9 Test Case 9.....	46
3.10 Test Case 10.....	48
3.11 Test Case 11.....	50
3.12 Test Case 12.....	52
3.13 Test Case 13.....	54
3.14 Test Case 14.....	56
3.15 Test Case 15.....	58
3.16 Test Case 16.....	58
3.17 Test Case 17.....	58
3.18 Test Case 18.....	59
3.19 Test Case 19.....	60
3.20 Test Case 20.....	60
3.21 Test Case 21.....	61
3.22 Test Case 22.....	61
3.23 Test Case 23.....	62
3.24 Test Case 24.....	62
BAB IV	
KESIMPULAN DAN SARAN.....	64
4.1 Kesimpulan.....	64
4.2 Saran.....	64
LAMPIRAN.....	66
REFERENSI.....	67

DAFTAR TABEL

Tabel 3.1 Input/Output Test Case 1.....	26
Tabel 3.2 Input/Output Test Case 2.....	29
Tabel 3.3 Input/Output Test Case 3.....	31
Tabel 3.4 Input/Output Test Case 4.....	34
Tabel 3.5 Input/Output Test Case 5.....	36
Tabel 3.6 Input/Output Test Case 6.....	39
Tabel 3.7 Input/Output Test Case 7.....	41
Tabel 3.8 Input/Output Test Case 8.....	43
Tabel 3.9 Input/Output Test Case 9.....	45
Tabel 3.10 Input/Output Test Case 10.....	47
Tabel 3.11 Input/Output Test Case 11.....	49
Tabel 3.12 Input/Output Test Case 12.....	51
Tabel 3.13 Input/Output Test Case 13.....	53
Tabel 3.14 Input/Output Test Case 14.....	54
Tabel 3.13 Input/Output Test Case 15.....	55
Tabel 3.16 Input/Output Test Case 16.....	55
Tabel 3.17 Input/Output Test Case 17.....	56
Tabel 3.18 Input/Output Test Case 18.....	57
Tabel 3.19 Input/Output Test Case 19.....	57
Tabel 3.20 Input/Output Test Case 20.....	58
Tabel 3.21 Input/Output Test Case 21.....	58
Tabel 3.22 Input/Output Test Case 22.....	59
Tabel 3.23 Input/Output Test Case 23.....	59
Tabel 3.24 Input/Output Test Case 24.....	60

DAFTAR GAMBAR

Gambar 2.2.1 (1) Kode addTab.....	16
Gambar 2.2.1 (2) Kode lookup.....	17
Gambar 2.2.2 Kode struct ASTNode.....	18
Gambar 2.2.3 (1) Kode buildAST.....	19
Gambar 2.2.3 (2) Kode buildFactor.....	20
Gambar 2.2.4 Kode visitProgram.....	21
Gambar 2.2.5 Kode visitVarDecl.....	22
Gambar 2.2.6 Kode visitProDecl.....	23
Gambar 2.2.7 Kode visitTypeDecl.....	24
Gambar 2.2.8 Kode visitBinOp.....	25
Gambar 2.2.9 Kode visitAssign.....	26

BAB I

DASAR TEORI

1.1 Semantic Analysis

Semantic Analysis merupakan tahap ketiga dalam pipeline compiler, dikerjakan setelah Lexical Analysis (tokenisasi) dan Syntax Analysis (pembentukan parse tree). Setelah parser menghasilkan parse tree yang membuktikan kebenaran sintaksis, semantic analyser bertugas memastikan program bermakna secara semantik, yaitu apakah konstruksi yang secara sintaksis valid juga masuk akal secara logika program.

Terdapat empat kategori pengecekan utama yang dilakukan oleh semantic analyser:

- Type Checking: memastikan setiap operator diterapkan pada operand dengan tipe yang kompatibel, misalnya tidak menjumlahkan nilai bertipe Boolean dengan Integer.
- Symbol Table Management: memastikan setiap identifier (variabel, fungsi, prosedur) telah dideklarasikan sebelum digunakan dan tidak dideklarasikan ulang dalam scope yang sama.
- Scope Resolution: menentukan validitas akses variabel berdasarkan hierarki blok kode, yaitu mekanisme lexical scoping dari scope terdalam ke terluar.
- Control Flow Validation: memastikan alur kontrol logis, misalnya break/continue hanya ada dalam loop, fungsi memiliki return di semua cabang, dan warning untuk unreachable code.

1.2 Abstract Syntax Tree (AST)

Abstract Syntax Tree (AST) adalah representasi ringkas dari parse tree. Berbeda dengan parse tree yang struktur nodenya mengikuti aturan grammar untuk keperluan pengecekan sintaksis, AST hanya menyimpan informasi yang relevan untuk tahap analisis dan pemrosesan selanjutnya.

Informasi sintaksis yang dihilangkan pada AST antara lain tanda kurung berlebih, keyword terminal seperti BEGIN, END, dan SEMICOLON, serta node-node non-terminal perantara seperti <expression>, <term>, dan <factor> ketika hanya memiliki satu child. Yang dipertahankan adalah hubungan semantik antar operator dan operand, yang direpresentasikan dalam node-node seperti AssignNode, BinOpNode, VarRefNode, dan sebagainya.

Proses pembentukan AST dari parse tree dilakukan melalui Syntax-Directed Translation (SDT). Setiap production rule grammar dilengkapi dengan semantic action yang mendeskripsikan

cara membentuk node AST yang sesuai. Atribut bersifat synthesized, artinya nilai atribut suatu node dihitung berdasarkan atribut child-nya (bottom-up). Berikut adalah contoh semantic rule yang digunakan dalam implementasi ini:

Production Rule	Semantic Rule	Keterangan
<assignment-statement> → ID := <expr>	assignment.node = AssignNode(VarNode(ID), expr.node)	Membentuk node penugasan dengan target dan nilai.
<factor> → INTCON	factor.node = ConstIntNode(INTCON.value)	Konstanta integer menjadi leaf node.
<factor> → TRUESY / FALSESY	factor.node = ConstBoolNode(true/false)	Literal boolean menjadi leaf node.
<expression> → <simple-expr> relop <term>	expr.node = BinOpNode(relop, left.node, right.node)	Operator relasional menghasilkan node BinOp bertipe Boolean.
<term> → <factor>	term.node = factor.node	Propagasi synthesized attribute ke atas.
<proc-call> → ID (<params>)	node = ProcCallNode(ID, params.nodes)	Pemanggilan prosedur/fungsi.

1.2.1 Implementasi Token

Decorated AST (atau Annotated AST) adalah AST yang telah diperkaya dengan informasi semantik melalui proses dekorasi. Proses ini dilakukan via penelusuran DFS (Depth-First Search) menggunakan fungsi visit pada setiap node AST.

Anotasi yang ditambahkan pada setiap node mencakup: tipe data akhir ekspresi (dataType), indeks entri di symbol table (tabIndex), lexical level atau scope tempat node berada (lexLevel), dan indeks block di btab (blockIndex) untuk node yang membuka scope baru.

1.3 Symbol Table

Symbol table adalah struktur data sentral yang digunakan semantic analyser untuk menyimpan dan mengambil informasi tentang setiap identifier yang dideklarasikan dalam program. Implementasi ini menggunakan tiga tabel yang saling melengkapi:

1.3.1 tab - Identifier Table

Tabel tab menyimpan satu baris per identifier. Atribut-atributnya adalah:

Atribut	Deskripsi
identifiers (name)	Nama identifier. Indeks user mulai dari 33 karena 0–32 untuk reserved words dan predefined identifiers.
link	Pointer/indeks ke identifier sebelumnya dalam scope yang sama (linked list per blok).
obj	Kelas objek: CONSTANT, VARIABLE, TYPE_NAME, PROCEDURE, FUNCTION, PROGRAM, FIELD, PARAMETER.
type	Tipe dasar identifier: INTEGER, REAL, CHAR, BOOLEAN, STRING, ARRAY, RECORD, dll.
ref	Pointer ke tabel lain jika tipe komposit: ke atab untuk array, ke btab untuk record/procedure.
nrm	1 = variabel normal; 0 = parameter by-reference (var parameter).
lev	Lexical level deklarasi: 0 = global, 1 = dalam procedure/function, dst.
adr	Offset variabel di stack frame, nilai konstanta, atau alamat procedure.

1.3.2 btab - Identifier Table

Tabel btab menyimpan satu baris per blok scope (prosedur, fungsi, atau definisi tipe record). Atribut-atributnya adalah:

Atribut	Deskripsi
blocks (idx)	Indeks entri blok.
last	Indeks identifier terakhir yang dideklarasikan dalam blok ini (ujung linked list tab).
lpar	Indeks parameter terakhir dari prosedur/fungsi. Bernilai 0 untuk blok record.
psze	Total ukuran parameter block dalam satuan memori.

Atribut	Deskripsi
vsze	Total ukuran variabel lokal block dalam satuan memori.

1.3.3 atab - Identifier Table

Tabel atab menyimpan detail tipe array. Atribut-atributnya adalah:

Atribut	Deskripsi
xtyp	Tipe indeks array (harus simple type bukan Real).
etyp	Tipe elemen array.
eref	Pointer ke atab atau btab jika elemen adalah tipe komposit.
low	Batas bawah indeks array.
high	Batas atas indeks array.
elsz	Ukuran satu elemen dalam satuan memori.
size	Total ukuran array = $(\text{high} - \text{low} + 1) \times \text{elsz}$.

1.4 Predefined Identifiers

Sebelum proses analisis dimulai, symbol table diinisialisasi dengan predefined identifiers yang otomatis tersedia tanpa perlu dideklarasikan oleh programmer. Identifiers ini menempati indeks 1–11 dalam tab.

Identifier	Keterangan
integer (idx=1)	TYPE_NAME bertipe INTEGER.
real (idx=2)	TYPE_NAME bertipe REAL.
char (idx=3)	TYPE_NAME bertipe CHAR.
boolean (idx=4)	TYPE_NAME bertipe BOOLEAN.
string (idx=5)	TYPE_NAME bertipe STRING.
true (idx=6)	CONSTANT bertipe BOOLEAN, adr=1 (nilai true). Termasuk reserved keyword.
false (idx=7)	CONSTANT bertipe BOOLEAN, adr=0 (nilai false). Termasuk reserved keyword.
writeln (idx=8)	PROCEDURE predefined untuk output dengan newline.
readln (idx=9)	PROCEDURE predefined untuk input.

Identifier	Keterangan
write (idx=10)	PROCEDURE predefined untuk output tanpa newline.
read (idx=11)	PROCEDURE predefined untuk input.

1.5 Type Compatibility

Type compatibility rule menentukan bagaimana tipe data dapat berinteraksi dalam ekspresi dan statement. Implementasi Arion Compiler menggunakan aturan-aturan berikut:

- Aturan Kompatibilitas Operator

Operator	Tipe Operand	Tipe Hasil
+, -, *	Integer atau Real	Integer atau Real (Real jika salah satu Real)
/	Integer atau Real	Selalu Real (real division)
div, mod	Integer saja	Integer
=, <, <=, >, >=, >	Integer, Real, Char, String	Boolean
and, or	Boolean saja	Boolean
not	Boolean saja	Boolean

- Assignment Compatibility

Sebuah nilai bertipe T2 dikatakan assignment-compatible dengan variabel bertipe T1 jika salah satu kondisi berikut terpenuhi:

1. T1 dan T2 adalah tipe yang sama persis.
2. T1 adalah Real dan T2 adalah Integer (implicit widening conversion).
3. T1 dan T2 adalah compatible dan salah satunya adalah SUBRANGE dari yang lain.
4. T1 dan T2 adalah compatible dan bertipe String.

1.6 Scope dan Lexical Level

Bahasa Arion menggunakan lexical scoping (static scoping). Setiap procedure atau function membentuk blok scope baru dengan lexical level yang lebih tinggi. Implementasi scope menggunakan struktur display: sebuah array yang menyimpan indeks btab dari setiap level aktif.

Saat memasuki blok baru, `enterBlock()` dipanggil, membuat entri btab baru dan menambahkannya ke display. Saat keluar dari blok, `leaveBlock()` dipanggil, membuang level terdalam dari display. Pencarian identifier (lookup) dilakukan dari level terdalam ke terluar, memungkinkan variable shadowing.

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1 Arsitektur Program

Dalam implementasi milestone 3, ditambahkan tiga modul baru ke atas fondasi Milestone 1 (lexer) dan Milestone 2 (parser). Program ditulis dalam bahasa C++ dan seluruh modul dihubungkan melalui file main.cpp. Berikut ini adalah struktur program dan fungsi tiap file.

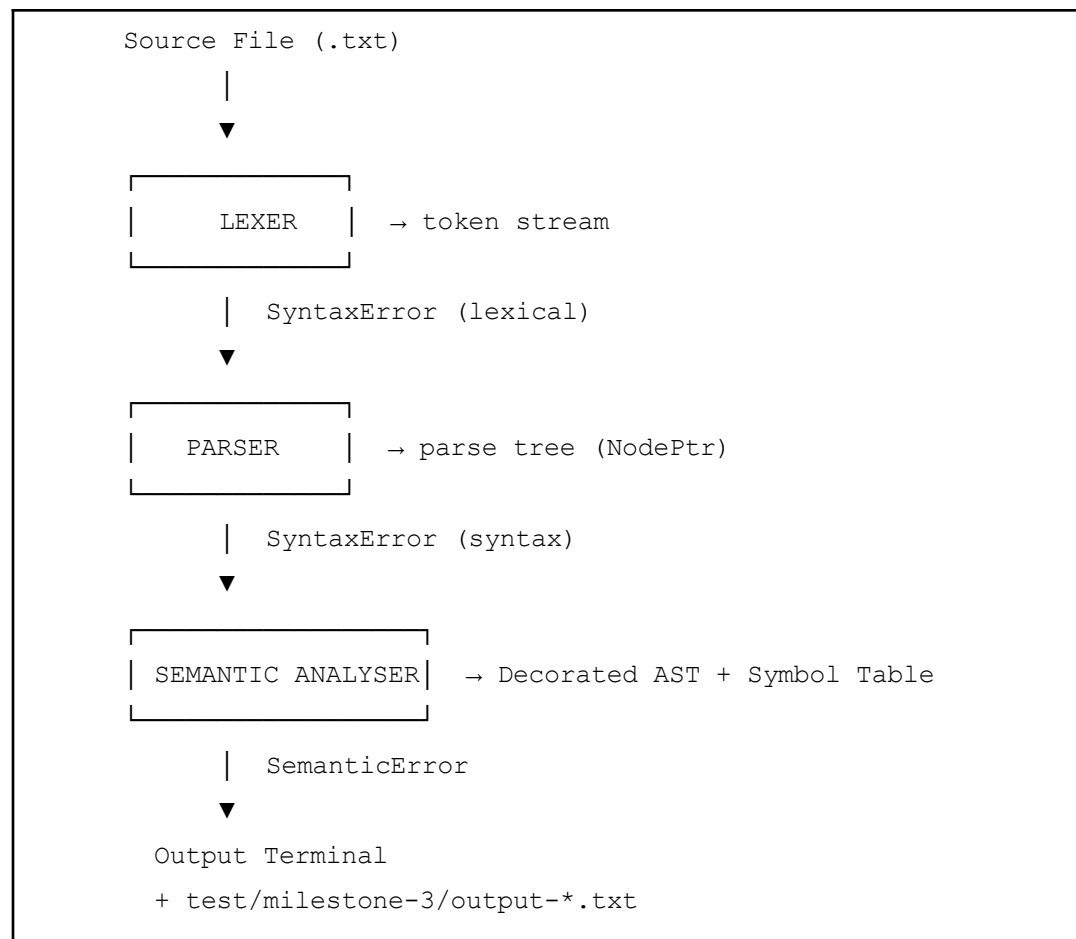
File	Peran
lexer/lexer.hpp & .cpp	DFA-based lexical analyser. Token TRUESY & FALSESY ditambahkan. Operator = (EQL) diperbaiki.
lexer/token.hpp & .cpp	Definisi enum TokenType dan konverter ke string.
parser/parser.hpp & .cpp	Recursive Descent Parser. parseFieldList, parseWhileStatement, parseForStatement diperbarui.
semantic/ast.hpp & .cpp	Definisi enum ASTNodeKind, DataType, struct ASTNode, dan fungsi printAST.
semantic/symbol_table.hpp & .cpp	Implementasi tiga symbol table (tab, btab, atab) dan operasi CRUD-nya.
semantic/semantic.hpp & .cpp	Semantic analyser utama: buildAST (fase 1) dan visitXxx (fase 2).
main.cpp	Entry point program. Memanggil lexer → parser → semantic analyser secara berurutan.

Tabel 2.1 Penjelasan Setiap File

2.1.1 Alur Eksekusi Program

Secara keseluruhan, program berjalan melalui tiga tahap besar yang saling berurutan. Jika salah satu tahap menghasilkan error, proses dihentikan dan pesan error ditampilkan ke terminal serta disimpan ke file output. Alur eksekusi program adalah sebagai berikut:

1. Lexer membaca source file dan menghasilkan token stream.
2. Parser mengonsumsi token stream dan menghasilkan parse tree.
3. `SemanticAnalyser.analyse(parseTree)` dipanggil.
4. Parse tree dikonversi menjadi AST menggunakan Syntax-Directed Translation.
5. AST ditelusuri secara DFS untuk mengisi symbol table, mengecek tipe, scope, dan mendekorasi setiap node.
6. Decorated AST dicetak ke terminal beserta isi tab, btab, dan atab.



2.1.2 Dependensi Antar Modul

Setiap modul hanya bergantung pada modul di bawahnya, sehingga perubahan pada satu modul tidak memengaruhi modul yang lain secara tidak terduga. Struktur dependensi modul program adalah sebagai berikut:

```
main.cpp
├─ lexer/lexer.hpp      (independen, tidak bergantung modul lain)
├─ parser/parser.hpp    (bergantung pada lexer/token.hpp)
└─ semantic/semantic.hpp (bergantung pada parser/parser.hpp,
                           semantic/ast.hpp,
                           semantic/symbol_table.hpp)
```

Modul `semantic` tidak perlu mengetahui cara kerja `lexer` secara internal. Ia hanya menerima `NodePtr` (hasil parse tree) dari `parser` sebagai input, lalu menghasilkan `ASTPtr` (Decorated AST) sebagai output.

2.1.3 Dua Fase Semantic Analyser

Inti dari Milestone 3 terdapat pada kelas `SemanticAnalyser`. Ketika fungsi `analyse(parseTree)` dipanggil, ia menjalankan dua fase secara berurutan, yaitu Syntax-Directed Translation dan Dekorasi AST.

1. Fase 1: Syntax-Directed Translation (Parse Tree → AST)

Fase 1 dilakukan oleh fungsi `buildAST()` beserta fungsi-fungsi `buildXxx()` pendampingnya. Tujuannya adalah mengubah parse tree yang verbose menjadi AST yang lebih ringkas dan mudah dianalisis. Node-node yang dihilangkan pada fase ini meliputi keyword terminal yang tidak membawa informasi semantik seperti `BEGIN`, `END`, `SEMICOLON`, `OF`, `DO`, `THEN`, tanda kurung berlebih pada ekspresi, serta node non-terminal perantara seperti `<term>` dan `<factor>` yang hanya memiliki satu child.

Sebagai contoh, ekspresi `a + 10` dalam parse tree memiliki beberapa lapisan node `<expression>` → `<simple-expression>` → `<term>` → `<factor>`. Setelah Fase 1, seluruh lapisan tersebut diratakan menjadi satu node `BinOp('+')` dengan dua child: `VarRef('a')` dan `ConstInt('10')`.

2. Fase 2: Dekorasi AST (Type Checking dan Symbol Table)

Fase ini dilakukan oleh fungsi `visitProgram()` yang memulai penelusuran DFS (Depth-First Search) pada AST hasil Fase 1. Setiap node AST dikunjungi tepat satu kali. Pada saat itulah identifier didaftarkan ke symbol table, setiap penggunaan identifier dicari melalui lookup, tipe data setiap ekspresi ditentukan dan dianotasikan ke node, kompatibilitas tipe diperiksa untuk setiap operasi dan assignment, serta scope dikelola dengan memanggil `enterBlock()` saat masuk blok baru dan `leaveBlock()` saat keluar.

	Fase 1	Fase 2
Nama	Syntax-Directed Translation	Dekorasi AST
Fungsi utama	<code>buildAST()</code> , <code>buildXxx()</code>	<code>visitProgram()</code> , <code>visitXxx()</code>
Input	Parse Tree (NodePtr)	AST bersih (ASTPtr)
Output	AST bersih (ASTPtr)	Decorated AST + Symbol Table
Tugas	Transformasi struktural	Analisis semantik dan anotasi

Tabel 2.1.3 Dua Fase Semantic Analyser

2.1.4 Justifikasi Pemilihan Implementasi

Beberapa keputusan desain diambil selama implementasi Milestone 3 dengan pertimbangan sebagai berikut.

Keputusan Desain	Justifikasi
Tiga tabel terpisah (tab, btab, atab)	Mengikuti arsitektur compiler Pascal klasik (Wirth). Setiap tabel memiliki tanggung jawab yang jelas sehingga penambahan tipe data baru hanya perlu dilakukan di atab tanpa mengubah struktur tab.

Linked list per blok di tab	Setiap entri tab menyimpan field link ke identifier sebelumnya dalam blok yang sama. Pengecekan redeklaras (lookupCurrentBlock) hanya menelusuri identifier dalam blok saat ini, bukan seluruh tabel, sehingga lebih efisien.
Display array untuk manajemen scope	display_[level] menyimpan indeks btab dari blok aktif pada setiap lexical level. Lookup berjalan dari level terdalam ke terluar, mendukung variable shadowing secara alami.
Pemisahan Fase 1 dan Fase 2	Fase 1 bersifat transformasi struktural murni tanpa state global sehingga mudah di-debug secara independen. Fase 2 menerima AST yang sudah bersih sehingga logika type checking dan scope tidak tercampur dengan logika parsing.

Tabel 2.1.4 Justifikasi Pilihan Implementasi

2.2 Implementasi Kode

2.2.1 Symbol Table (symbol_table.hpp / .cpp)

Kelas SymbolTable mengenkapsulasi ketiga tabel (tab_, btab_, atab_) dan mekanisme scope via display_. Konstruktor memanggil initPredefined() yang mengisi tab_[0..32] sampai index 0 adalah dummy, 1–11 adalah predefined identifiers, dan 12–32 adalah padding agar user identifier selalu mulai dari indeks 33. Berikut ini adalah

mekanisme

per

blok:



```
1  int SymbolTable::addTab(const std::string& name, ObjClass obj,
2                          DataType type, int ref, int nrm, int adr) {
3      TabEntry e;
4      e.name = name;
5      e.obj  = obj;
6      e.type = type;
7      e.ref  = ref;
8      e.nrm  = nrm;
9      e.lev  = currentLevel();
10     e.adr  = adr;
11
12     // link ke identifier sebelumnya
13     int curBlock = display_.back();
14     e.link = btab[curBlock].last;
15
16     e.initialized = false;
17     tab_.push_back(e);
18     int newIdx = (int)tab_.size() - 1;
19
20     // update last di current blok
21     btab[curBlock].last = newIdx;
22
23     return newIdx;
24 }
```

Gambar 2.2.1 (1) Kode addTab

Setiap identifier baru menjadi ujung (head) linked list di blok saat ini melalui field link. Ini memungkinkan lookup per-blok berjalan efisien hanya dengan mengikuti chain link dari btab[block].last.

Fungsi lookup() menelusuri dari level terdalam ke terluar untuk mendukung variable shadowing:

```

1  int SymbolTable::lookup(const std::string& name) const {
2      std::string lower = name;
3      std::transform(lower.begin(), lower.end(), lower.begin(), ::tolower);
4
5      // Setiap level ditelusurin
6      for (int lv = currentLevel(); lv >= 0; --lv) {
7          int blockIdx = display_[lv];
8          // Mengikuti linked list yang ada di dalam blok
9          int idx = btab_[blockIdx].last;
10         while (idx > 0) {
11             std::string entryName = tab_[idx].name;
12             std::transform(entryName.begin(), entryName.end(),
13                             entryName.begin(), ::tolower);
14             if (entryName == lower) return idx;
15             idx = tab_[idx].link;
16         }
17     }
18
19     for (int i = 0; i < TAB_USER_START; ++i) {
20         std::string entryName = tab_[i].name;
21         std::transform(entryName.begin(), entryName.end(),
22                         entryName.begin(), ::tolower);
23         if (!entryName.empty() && entryName == lower) return i;
24     }
25
26     return -1;
27 }

```

Gambar 2.2.1 (2) Kode lookup

2.2.2 Definisi AST Node (ast.hpp)

File ast.hpp mendefinisikan enum DataType (tipe data yang diketahui semantic analyser) dan enum ASTNodeKind (jenis-jenis node AST). Struct ASTNode menyimpan kind, name, children, serta anotasi semantik.

```

1  struct ASTNode {
2      ASTNodeKind kind;
3      std::string name;
4      std::vector<std::shared_ptr<ASTNode>> children;
5
6      DataType dataType = DataType::NOTYPE; // tipe hasil ekspresi / deklarasi
7      int tabIndex = -1; // indeks di tab
8      int blockIndex = -1; // indeks di btab
9      int arrayIndex = -1; // indeks di atab
10     int lexLevel = -1; // lexical level
11     bool initialized = false;
12
13     explicit ASTNode(ASTNodeKind k, const std::string& n = "")
14         : kind(k), name(n) {}
15 };

```

Gambar 2.2.2 Kode struct ASTNode

Setiap field anotasi diisi oleh fungsi visit yang bersesuaian. Misalnya, tabIndex diisi oleh visitVarRef, visitProcDecl, visitFuncDecl, dll. Field dataType diisi oleh semua fungsi visitExpr.

2.2.3 buildAST : Syntax-Directed Translation (semantic.cpp)

Fungsi buildAST adalah dispatcher utama Fase 1. Ia menerima sebuah node parse tree dan mengembalikan node AST yang sesuai berdasarkan label node. Setiap label production rule dipetakan ke fungsi build yang khusus.

```

1  ASTPtr SemanticAnalyser::buildAST(const NodePtr& n) {
2      if (!n) return nullptr;
3      const std::string& lbl = n->label;
4
5      if (lbl == "<program>")          return buildProgram(n);
6      if (lbl == "<declaration-part>") return buildDeclPart(n);
7      if (lbl == "<const-declaration>") return buildConstDecl(n);
8      if (lbl == "<var-declaration>")   return buildVarDecl(n);
9      if (lbl == "<type-declaration>")  return buildTypeDecl(n);
10     if (lbl == "<subprogram-declaration>") return buildSubprogDecl(n);
11     if (lbl == "<procedure-declaration>") return buildProcDecl(n);
12     if (lbl == "<function-declaration>") return buildFuncDecl(n);
13     if (lbl == "<block>")              return buildBlock(n);
14     if (lbl == "<compound-statement>") return buildCompound(n);
15     if (lbl == "<statement-list>")     return buildStmtList(n);
16     if (lbl == "<statement>")         return buildStatement(n);
17     if (lbl == "<assignment-statement>") return buildAssign(n);
18     if (lbl == "<if-statement>")       return buildIfStmt(n);
19     if (lbl == "<while-statement>")    return buildWhileStmt(n);
20     if (lbl == "<for-statement>")      return buildForStmt(n);
21     if (lbl == "<repeat-statement>")   return buildRepeatStmt(n);

```

Gambar 2.2.3 (1) Kode buildAST

Contoh konkret: fungsi buildFactor() yang mengonversi leaf parse tree menjadi node AST:

```

1  ASTPtr SemanticAnalyser::buildFactor(const NodePtr& n) {
2      if (n->children.empty()) return makeNode(ASTNodeKind::EMPTY_STMT);
3
4      auto& first = n->children[0];
5      const std::string& lbl = first->label;
6
7      // Literal
8      if (labelStartsWith(first, "INTCON")) return makeNode(ASTNodeKind::CONST_INT,    valueOf(first));
9      if (labelStartsWith(first, "REALCON")) return makeNode(ASTNodeKind::CONST_REAL,  valueOf(first));
10     if (labelStartsWith(first, "CHARCON")) return makeNode(ASTNodeKind::CONST_CHAR,  valueOf(first));
11     if (labelStartsWith(first, "STRING"))  return makeNode(ASTNodeKind::CONST_STRING, valueOf(first));
12     if (labelStartsWith(first, "TRUESY"))  return makeNode(ASTNodeKind::CONST_BOOL,  "true");
13     if (labelStartsWith(first, "FALSESY")) return makeNode(ASTNodeKind::CONST_BOOL,  "false");
14
15     // Variabel atau function call
16     if (lbl == "<variable>") return buildVariable(first);
17     if (lbl == "<procedure/function-call>") {
18         // function call dalam ekspresi
19         auto fc = buildProcCall(first);
20         fc->kind = ASTNodeKind::FUNC_CALL;
21         return fc;
22     }
23
24     // Ekspresi dalam kurung ( <expression> )
25     if (labelStartsWith(first, "LPARENT")) {
26         if (n->children.size() >= 2)
27             return buildExpression(n->children[1]);
28     }
29
30     // NOT <factor>
31     if (labelStartsWith(first, "NOTSY")) {
32         auto notNode = makeNode(ASTNodeKind::UNOP, "not");
33         if (n->children.size() >= 2)
34             notNode->children.push_back(buildFactor(n->children[1]));
35         return notNode;
36     }
37
38     return makeNode(ASTNodeKind::EMPTY_STMT);
39 }

```

Gambar 2.2.3 (2) Kode buildFactor

2.2.4 visitProgram: Dekorasi AST (semantic.cpp)

Proses ini dimulai dari fungsi visitProgram yang menjadi entry point DFS traversal. Fungsi ini membuat global block (btabs[0]), mendaftarkan nama program ke tab, lalu mendelegasikan ke visitBlock untuk menangani declarations dan compound statement.

```

1 void SemanticAnalyser::visitProgram(const ASTPtr& n) {
2     if (!n) return;
3
4     // Masukkan nama program ke tab
5     int idx = symtab_.addTab(n->name, ObjClass::PROGRAM, DataType::VOID);
6     n->tabIndex = idx;
7     n->lexLevel = 0;
8     n->dataType = DataType::VOID;
9
10    // Deklarasi global tetap di level 0
11    for (auto& ch : n->children) {
12        if (!ch) continue;
13        if (ch->kind == ASTNodeKind::BLOCK && ch->name == "declarations") {
14            visitBlock(ch);
15        }
16    }
17
18    // Body program dijalankan dalam blok sendiri (main block)
19    int mainBlockIdx = symtab_.enterBlock();
20    n->blockIndex = mainBlockIdx;
21
22    for (auto& ch : n->children) {
23        if (!ch) continue;
24        if (ch->kind == ASTNodeKind::COMPOUND) {
25            ch->blockIndex = mainBlockIdx;
26            ch->lexLevel = symtab_.currentLevel();
27            visitCompound(ch);
28        } else if (ch->kind == ASTNodeKind::BLOCK && ch->name != "declarations") {
29            ch->blockIndex = mainBlockIdx;
30            ch->lexLevel = symtab_.currentLevel();
31            visitBlock(ch);
32        }
33    }
34
35    symtab_.leaveBlock();
36 }

```

Gambar 2.2.4 Kode visitProgram

2.2.5 visitVarDecl: Registrasi Variabel (semantic.cpp)

Penanganan tipe record merupakan salah satu implementasi yang paling kompleks. Ketika ditemukan deklarasi type `TPoint = record...`, fungsi `visitTypeDecl` membuat blok `btabs` khusus untuk menampung field-field record, mendaftarkan setiap field ke tab dalam blok tersebut, lalu menyimpan indeks blok tersebut (`ref`) di entri tab untuk `TPoint`.

```

1 void SemanticAnalyser::visitVarDecl(const ASTPtr& n) {
2     if (n->name.empty()) {
3         for (auto& ch : n->children) {
4             if (ch && ch->kind == ASTNodeKind::VAR_DECL) visitVarDecl(ch);
5         }
6         return;
7     }
8
9     ASTPtr legacyIdentList = nullptr;
10    ASTPtr legacyTypeNode = nullptr;
11    for (auto& ch : n->children) {
12        if (!ch) continue;
13        if (ch->kind == ASTNodeKind::IDENT_LIST) legacyIdentList = ch;
14        else if (!legacyTypeNode) legacyTypeNode = ch;
15    }
16    if (legacyIdentList) {
17        DataType dt = resolveTypeNode(legacyTypeNode);
18        int ref = 0;
19        int adrOffset = symtab_.btab()[symtab_.blockAt(symtab_.currentLevel())].vsze;
20        for (auto& identNode : legacyIdentList->children) {
21            std::string name = identNode->name;
22            if (symtab_.lookupCurrentBlock(name) >= 0)
23                semanticError("Identifier '" + name + "' sudah dideklarasikan dalam scope ini.");
24            int varSize = symtab_.sizeof(dt, ref);
25            int tabIdx = symtab_.addTab(name, ObjClass::VARIABLE, dt, ref, 1, adrOffset);
26            adrOffset += varSize;
27            identNode->tabIndex = tabIdx;
28            identNode->lexLevel = symtab_.currentLevel();
29            identNode->dataType = dt;
30            symtab_.btab()[symtab_.blockAt(symtab_.currentLevel())].vsze = adrOffset;
31        }
32        n->dataType = dt;
33        return;
34    }
35 }

```

Gambar 2.2.5 Kode visitVarDecl

2.2.6 visitProcDecl / visitFuncDecl: Registrasi Subprogram (semantic.cpp)

Fungsi visitProcDecl dan visitFuncDecl menangani registrasi prosedur/fungsi beserta parameter formalnya. Prosedur/fungsi didaftarkan ke tab sebelum masuk blok agar bisa rekursif. Setelah masuk blok baru (enterBlock), parameter formal didaftarkan ke tab di dalam blok tersebut, dan lpar serta psze di btab diupdate.

```

1 void SemanticAnalyser::visitProcDecl(const ASTPtr& n) {
2     if (symtab_.lookupCurrentBlock(n->name) >= 0)
3         semanticError("Procedure '" + n->name + "' sudah dideklarasikan.");
4
5     // Mendaftarkan procedure ke tab
6     int procIdx = symtab_.addTab(n->name, ObjClass::PROCEDURE, DataType::VOID);
7     n->tabIndex = procIdx;
8     n->dataType = DataType::VOID;
9
10    // Masuk ke blok baru
11    int blockIdx = symtab_.enterBlock();
12    n->blockIndex = blockIdx;
13    n->lexLevel = symtab_.currentLevel();
14
15    for (auto& ch : n->children) {
16        if (ch->kind == ASTNodeKind::IDENT_LIST && ch->name == "params") {
17            int adrOffset = 0;
18            for (auto& pg : ch->children) {
19                if (pg->kind != ASTNodeKind::PARAM_DECL) continue;
20                ASTPtr identList = nullptr;
21                ASTPtr typeNode = nullptr;
22                for (auto& pgch : pg->children) {
23                    if (pgch->kind == ASTNodeKind::IDENT_LIST) identList = pgch;
24                    else typeNode = pgch;
25                }
26                if (!identList) continue;
27                DataType dt = resolveTypeNode(typeNode);
28                for (auto& idNode : identList->children) {
29                    if (symtab_.lookupCurrentBlock(idNode->name) >= 0)
30                        semanticError("Parameter '" + idNode->name + "' sudah dideklarasikan.");
31                    int varSize = symtab_.sizeOf(dt);
32                    int tabIdx = symtab_.addTab(idNode->name, ObjClass::PARAMETER, dt, 0, 1, adrOffset);
33                    symtab_.tab()[tabIdx].initialized = true;
34                    adrOffset += varSize;
35                    idNode->tabIndex = tabIdx;
36                    idNode->lexLevel = symtab_.currentLevel();
37                    idNode->dataType = dt;
38                }
39            }
40            int curBlock = symtab_.blockAt(symtab_.currentLevel());
41            symtab_.btab()[curBlock].lpar = symtab_.btab()[curBlock].last;
42            symtab_.btab()[curBlock].psize = adrOffset;
43        } else if (ch->kind == ASTNodeKind::BLOCK) {
44            visitBlock(ch);
45        } else if (ch->kind == ASTNodeKind::COMPOUND) {
46            visitCompound(ch);
47        }
48    }
49
50    symtab_.leaveBlock();
51 }

```

Gambar 2.2.6 Kode visitProDecl

2.2.7 visitTypeDecl: Registrasi Tipe Record (semantic.cpp)

Penanganan tipe record merupakan salah satu implementasi yang paling kompleks. Ketika ditemukan deklarasi tipe `TPoint = record...`, fungsi `visitTypeDecl` membuat blok `btab` khusus untuk menampung field-field record, mendaftarkan setiap field ke tab dalam blok tersebut, lalu menyimpan indeks blok tersebut (`ref`) di entri tab untuk `TPoint`.

```

1 void SemanticAnalyser::visitTypeDecl(const ASTPtr& n) {
2     if (n->name.empty()) {
3         for (auto& ch : n->children) visitTypeDecl(ch);
4         return;
5     }
6     if (symtab_.lookupCurrentBlock(n->name) >= 0)
7         semanticError("Tipe '" + n->name + "' sudah dideklarasikan.");
8
9     DataType dt = DataType::NOTYPE;
10    int ref = 0;
11
12    if (!n->children.empty() && n->children[0]->kind == ASTNodeKind::TYPE_RECORD) {
13        dt = DataType::RECORD;
14        int blockIdx = symtab_.enterBlock();
15        ref = blockIdx;
16        auto& fieldList = n->children[0]->children;
17        for (auto& fieldBlock : fieldList) {
18            if (!fieldBlock) continue;
19            for (auto& fieldDecl : fieldBlock->children) {
20                if (!fieldDecl || fieldDecl->kind != ASTNodeKind::FIELD_DECL) continue;
21                ASTPtr identList = nullptr;
22                ASTPtr typeNode = nullptr;
23                for (auto& fc : fieldDecl->children) {
24                    if (fc->kind == ASTNodeKind::IDENT_LIST) identList = fc;
25                    else typeNode = fc;
26                }
27                if (!identList) continue;
28                DataType fdt = resolveTypeNode(typeNode);
29                int adr = symtab_.btabs()[blockIdx].vsze;
30                for (auto& idNode : identList->children) {
31                    int tabIdx = symtab_.addTab(idNode->name, ObjClass::FIELD, fdt, 0, 1, adr);
32                    adr += symtab_.sizeof(fdt);
33                    symtab_.btabs()[blockIdx].vsze = adr;
34                    idNode->tabIndex = tabIdx;
35                    idNode->dataType = fdt;
36                }
37            }
38        }
39        symtab_.leaveBlock();
40    } else if (!n->children.empty()) {
41        dt = resolveTypeNode(n->children[0]);
42    }
43
44    int idx = symtab_.addTab(n->name, ObjClass::TYPE_NAME, dt, ref);
45    n->tabIndex = idx;
46    n->dataType = dt;
47 }

```

Gambar 2.2.7 Kode visitTypeDecl

2.2.8 Type Checking: visitBinOp dan resultTypeOfBinOp (semantic.cpp)

Pengecekan tipe untuk operator biner dilakukan di visitBinOp yang memanggil resultTypeOfBinOp. Fungsi ini mengimplementasikan seluruh aturan type compatibility: operator aritmatika, relasional, dan logika.

```

1  DataType SemanticAnalyser::visitBinOp(const ASTPtr& n) {
2      if (n->children.size() < 2) return DataType::NOTYPE;
3      DataType lt = visitExpr(n->children[0]);
4      DataType rt = visitExpr(n->children[1]);
5      DataType result = resultTypeOfBinOp(n->name, lt, rt);
6      if (result == DataType::UNKNOWN)
7          semanticError("Operator '" + n->name + "' tidak bisa diterapkan pada tipe '"
8              + dataTypeToString(lt) + "' dan '" + dataTypeToString(rt) + "'.");
9      n->dataType = result;
10     return result;
11 }
12
13
14 DataType SemanticAnalyser::visitUnOp(const ASTPtr& n) {
15     if (n->children.empty()) return DataType::NOTYPE;
16     DataType dt = visitExpr(n->children[0]);
17     if (n->name == "-") {
18         if (dt != DataType::INTEGER && dt != DataType::REAL)
19             semanticError("Operator unary '-' hanya berlaku untuk tipe numerik.");
20         n->dataType = dt;
21         return dt;
22     }
23     if (n->name == "not") {
24         if (dt != DataType::BOOLEAN)
25             semanticError("Operator 'not' hanya berlaku untuk tipe Boolean.");
26         n->dataType = DataType::BOOLEAN;
27         return DataType::BOOLEAN;
28     }
29     n->dataType = dt;
30     return dt;
31 }
32

```

Gambar 2.2.8 Kode visitBinOp

2.2.9 visitAssign: Pengecekan Assignment Compatibility (semantic.cpp)

Pengecekan tipe untuk operator biner dilakukan di visitBinOp yang memanggil resultTypeOfBinOp. Fungsi ini mengimplementasikan seluruh aturan type compatibility: operator aritmatika, relasional, dan logika.

```

1 void SemanticAnalyser::visitAssign(const ASTPtr& n) {
2     if (n->children.size() < 2) return;
3
4     auto& target = n->children[0];
5     DataType targetType = DataType::NOTYPE;
6     if (target->kind == ASTNodeKind::VAR_REF) {
7         int idx = symtab_.lookup(target->name);
8         if (idx < 0)
9             semanticError("Identifier '" + target->name + "' belum dideklarasikan.");
10        auto& entry = symtab_.tab()[idx];
11        target->tabIndex = idx;
12        target->lexLevel = entry.lev;
13        target->dataType = entry.type;
14        targetType = entry.type;
15    } else {
16        targetType = visitExpr(target);
17    }
18
19    DataType valueType = visitExpr(n->children[1]);
20
21    if (!isAssignmentCompatible(targetType, valueType)) {
22        semanticError("Assignment tidak kompatibel: target bertipe '"
23            + dataTypeToString(targetType) + "', nilai bertipe '"
24            + dataTypeToString(valueType) + "'.");
25    }
26
27    if (target->tabIndex >= 0) {
28        symtab_.tab()[target->tabIndex].initialized = true;
29        target->initialized = true;
30    }
31
32    n->dataType = DataType::VOID;
33 }

```

Gambar 2.2.9 Kode visitAssign

BAB III

PENGUJIAN

3.1 Test Case 1

Masukan (input)
<pre>program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>
Keluaran (output)
DECORATED ABSTRACT SYNTAX TREE <pre>└─ Program('Hello') → type:Void tab:33 btab:1 lev:0 └─ Block('declarations') └─ VarDecl('a') → type:Integer tab:34 lev:0 └─ VarDecl('b') → type:Integer tab:35 lev:0 └─ Block → btab:1 lev:1 └─ Assign('a' := 5) → type:Void └─ VarRef('a') → type:Integer tab:34 lev:0 └─ ConstInt('5') → type:Integer └─ Assign('b' := 'a'+10) → type:Void └─ VarRef('b') → type:Integer tab:35 lev:0 └─ BinOp('+') → type:Integer └─ VarRef('a') → type:Integer tab:34 lev:0 └─ ConstInt('10') → type:Integer └─ writeln(...) → type:Void tab:8 lev:0 └─ ConstString('Result = ') → type:String └─ VarRef('b') → type:Integer tab:35 lev:0 └─ EmptyStmt</pre>

===== TAB (Identifier Table) =====								
idx	name	obj	type	ref	nrm	lev	adr	link

0		variable	NoType	0	1	0	0	0
1	integer	type	Integer	0	1	0	0	0
2	real	type	Real	0	1	0	0	0
3	char	type	Char	0	1	0	0	0
4	boolean	type	Boolean	0	1	0	0	0
5	string	type	String	0	1	0	0	0
6	true	constant	Boolean	0	1	0	1	0
7	false	constant	Boolean	0	1	0	0	0
8	writeln	procedure	Void	0	1	0	0	0
9	readln	procedure	Void	0	1	0	0	0
10	write	procedure	Void	0	1	0	0	0
11	read	procedure	Void	0	1	0	0	0
33	Hello	program	Void	0	1	0	0	0
34	a	variable	Integer	0	1	0	0	33
35	b	variable	Integer	0	1	0	1	34
===== BTAB (Block Table) =====								
idx	last	lpar	psze	vsze				

0	35	0	0	2				
1	0	0	0	0				
===== ATAB (Array Table) =====								
(kosong)								

Tabel 3.1 Input/Output Test Case 1

3.2 Test Case 2

Masukan (input)
<pre> program TestBasicTypes; var i: integer; r: real; c: char; b: boolean; </pre>

```

begin
  i := 42;
  r := 3.14;
  c := 'A';
  b := true;
  b := i > 10;
  b := i == 42;
  r := i;
  writeln(i);
  writeln(r);
end.

```

Keluaran (output)

DECORATED ABSTRACT SYNTAX TREE

```

└─ Program('TestBasicTypes') → type:Void tab:33 btab:1 lev:0
  └─ Block('declarations')
    └─ VarDecl('i') → type:Integer tab:34 lev:0
    └─ VarDecl('r') → type:Real tab:35 lev:0
    └─ VarDecl('c') → type:Char tab:36 lev:0
    └─ VarDecl('b') → type:Boolean tab:37 lev:0
  └─ Block → btab:1 lev:1
    └─ Assign('i' := 42) → type:Void
      └─ VarRef('i') → type:Integer tab:34 lev:0
      └─ ConstInt('42') → type:Integer
    └─ Assign('r' := 3.14) → type:Void
      └─ VarRef('r') → type:Real tab:35 lev:0
      └─ ConstReal('3.14') → type:Real
    └─ Assign('c' := A) → type:Void
      └─ VarRef('c') → type:Char tab:36 lev:0
      └─ ConstChar('A') → type:Char
    └─ Assign('b' := true) → type:Void
      └─ VarRef('b') → type:Boolean tab:37 lev:0
      └─ ConstBool('true') → type:Boolean
    └─ Assign('b' := 'i'>10) → type:Void
      └─ VarRef('b') → type:Boolean tab:37 lev:0
      └─ BinOp('>') → type:Boolean
        └─ VarRef('i') → type:Integer tab:34 lev:0
        └─ ConstInt('10') → type:Integer

```

```

└─ Assign('b' := 'i'=42) → type:Void
  └─ VarRef('b') → type:Boolean tab:37 lev:0
    └─ BinOp('=') → type:Boolean
      └─ VarRef('i') → type:Integer tab:34 lev:0
        └─ ConstInt('42') → type:Integer
└─ Assign('r' := 'i') → type:Void
  └─ VarRef('r') → type:Real tab:35 lev:0
    └─ VarRef('i') → type:Integer tab:34 lev:0
└─ writeln(...) → type:Void tab:8 lev:0
  └─ VarRef('i') → type:Integer tab:34 lev:0
└─ writeln(...) → type:Void tab:8 lev:0
  └─ VarRef('r') → type:Real tab:35 lev:0
└─ EmptyStmt

```

===== TAB (Identifier Table) =====

idx	name	obj	type	ref	nrm	lev	adr	link

0		variable	NoType	0	1	0	0	0
1	integer	type	Integer	0	1	0	0	0
2	real	type	Real	0	1	0	0	0
3	char	type	Char	0	1	0	0	0
4	boolean	type	Boolean	0	1	0	0	0
5	string	type	String	0	1	0	0	0
6	true	constant	Boolean	0	1	0	1	0
7	false	constant	Boolean	0	1	0	0	0
8	writeln	procedure	Void	0	1	0	0	0
9	readln	procedure	Void	0	1	0	0	0
10	write	procedure	Void	0	1	0	0	0
11	read	procedure	Void	0	1	0	0	0
33	TestBasicTypes	program	Void	0	1	0	0	0
34	i	variable	Integer	0	1	0	0	33
35	r	variable	Real	0	1	0	1	34
36	c	variable	Char	0	1	0	3	35
37	b	variable	Boolean	0	1	0	4	36

===== BTAB (Block Table) =====

idx	last	lpar	psze	vsze

0	37	0	0	5
1	0	0	0	0

===== ATAB (Array Table) =====

(kosong)

Tabel 3.2 Input/Output Test Case 2

3.3 Test Case 3

Masukan (input)
<pre> program TestConst; const MAX == 100; PI == 3.14; var x, y: integer; z: real; begin x := MAX; y := x + 5; y := x - 3; y := x * 2; z := x / 4; y := x div 3; y := x mod 7; z := PI; writeln(y); writeln(z); end. </pre>
Keluaran (output)
DECORATED ABSTRACT SYNTAX TREE <pre> └─ Program('TestConst') → type:Void tab:33 btab:1 lev:0 └─ Block('declarations') └─ ConstDecl └─ ConstDecl('MAX') → type:Integer tab:34 lev:0 └─ ConstInt('100') → type:Integer └─ ConstDecl('PI') → type:Real tab:35 lev:0 └─ ConstReal('3.14') → type:Real └─ VarDecl('x') → type:Integer tab:36 lev:0 </pre>

```

|   |— VarDecl('y') → type:Integer tab:37 lev:0
|   |— VarDecl('z') → type:Real tab:38 lev:0
|— Block → btab:1 lev:1
|   |— Assign('x' := 'MAX') → type:Void
|   |   |— VarRef('x') → type:Integer tab:36 lev:0
|   |   |— VarRef('MAX') → type:Integer tab:34 lev:0
|   |— Assign('y' := 'x'+5) → type:Void
|   |   |— VarRef('y') → type:Integer tab:37 lev:0
|   |   |— BinOp('+') → type:Integer
|   |       |— VarRef('x') → type:Integer tab:36 lev:0
|   |       |— ConstInt('5') → type:Integer
|   |— Assign('y' := 'x'-3) → type:Void
|   |   |— VarRef('y') → type:Integer tab:37 lev:0
|   |   |— BinOp('-') → type:Integer
|   |       |— VarRef('x') → type:Integer tab:36 lev:0
|   |       |— ConstInt('3') → type:Integer
|   |— Assign('y' := 'x'*2) → type:Void
|   |   |— VarRef('y') → type:Integer tab:37 lev:0
|   |   |— BinOp('*') → type:Integer
|   |       |— VarRef('x') → type:Integer tab:36 lev:0
|   |       |— ConstInt('2') → type:Integer
|   |— Assign('z' := 'x'/4) → type:Void
|   |   |— VarRef('z') → type:Real tab:38 lev:0
|   |   |— BinOp('/') → type:Real
|   |       |— VarRef('x') → type:Integer tab:36 lev:0
|   |       |— ConstInt('4') → type:Integer
|   |— Assign('y' := 'x'div3) → type:Void
|   |   |— VarRef('y') → type:Integer tab:37 lev:0
|   |   |— BinOp('div') → type:Integer
|   |       |— VarRef('x') → type:Integer tab:36 lev:0
|   |       |— ConstInt('3') → type:Integer
|   |— Assign('y' := 'x'mod7) → type:Void
|   |   |— VarRef('y') → type:Integer tab:37 lev:0
|   |   |— BinOp('mod') → type:Integer
|   |       |— VarRef('x') → type:Integer tab:36 lev:0
|   |       |— ConstInt('7') → type:Integer
|   |— Assign('z' := 'PI') → type:Void
|   |   |— VarRef('z') → type:Real tab:38 lev:0
|   |   |— VarRef('PI') → type:Real tab:35 lev:0
|   |— writeln(...) → type:Void tab:8 lev:0
|   |   |— VarRef('y') → type:Integer tab:37 lev:0
|   |— writeln(...) → type:Void tab:8 lev:0
|   |   |— VarRef('z') → type:Real tab:38 lev:0
|— EmptyStmt

```

===== TAB (Identifier Table) =====								
idx	name	obj	type	ref	nrm	lev	adr	link

0		variable	NoType	0	1	0	0	0
1	integer	type	Integer	0	1	0	0	0
2	real	type	Real	0	1	0	0	0
3	char	type	Char	0	1	0	0	0
4	boolean	type	Boolean	0	1	0	0	0
5	string	type	String	0	1	0	0	0
6	true	constant	Boolean	0	1	0	1	0
7	false	constant	Boolean	0	1	0	0	0
8	writeln	procedure	Void	0	1	0	0	0
9	readln	procedure	Void	0	1	0	0	0
10	write	procedure	Void	0	1	0	0	0
11	read	procedure	Void	0	1	0	0	0
33	TestConst	program	Void	0	1	0	0	0
34	MAX	constant	Integer	0	1	0	0	33
35	PI	constant	Real	0	1	0	0	34
36	x	variable	Integer	0	1	0	0	35
37	y	variable	Integer	0	1	0	1	36
38	z	variable	Real	0	1	0	2	37
===== BTAB (Block Table) =====								
idx	last	lpar	psze	vsze				

0	38	0	0	4				
1	0	0	0	0				
===== ATAB (Array Table) =====								
(kosong)								

Tabel 3.3 Input/Output Test Case 3

3.4 Test Case 4

Masukan (input)
<pre> program TestControl; var x, y: integer; flag: boolean; </pre>

```

begin
  x := 10;
  y := 20;
  flag := (x < y) and (y > 0);
  if flag then
    begin
      x := x + 1;
    end
  else
    begin
      x := x - 1;
    end;
  if not flag then
    begin
      y := 0;
    end;
  writeln(x);
end.

```

Keluaran (output)

DECORATED ABSTRACT SYNTAX TREE

```

└─ Program('TestControl') → type:Void tab:33 btab:1 lev:0
  └─ Block('declarations')
    └─ VarDecl('x') → type:Integer tab:34 lev:0
    └─ VarDecl('y') → type:Integer tab:35 lev:0
    └─ VarDecl('flag') → type:Boolean tab:36 lev:0
  └─ Block → btab:1 lev:1
    └─ Assign('x' := 10) → type:Void
      └─ VarRef('x') → type:Integer tab:34 lev:0
      └─ ConstInt('10') → type:Integer
    └─ Assign('y' := 20) → type:Void
      └─ VarRef('y') → type:Integer tab:35 lev:0
      └─ ConstInt('20') → type:Integer
    └─ Assign('flag' := 'x'<'y'and'y'>0) → type:Void
      └─ VarRef('flag') → type:Boolean tab:36 lev:0
      └─ BinOp('and') → type:Boolean
        └─ BinOp('<') → type:Boolean

```


8	writeln	procedure	Void	0	1	0	0	0
9	readln	procedure	Void	0	1	0	0	0
10	write	procedure	Void	0	1	0	0	0
11	read	procedure	Void	0	1	0	0	0
33	TestControl	program	Void	0	1	0	0	0
34	x	variable	Integer	0	1	0	0	33
35	y	variable	Integer	0	1	0	1	34
36	flag	variable	Boolean	0	1	0	2	35

===== BTAB (Block Table) =====

idx	last	lpar	psze	vsze

0	36	0	0	3
1	0	0	0	0

===== ATAB (Array Table) =====

(kosong)

Tabel 3.4 Input/Output Test Case 4

3.5 Test Case 5

Masukan (input)
<pre> program TestWhile; var i, sum: integer; begin sum := 0; i := 1; while i <= 5 do begin sum := sum + i; i := i + 1; end; end. </pre>
Keluaran (output)
<pre> </pre>

DECORATED ABSTRACT SYNTAX TREE

```

└─ Program('TestWhile') → type:Void tab:33 btab:1 lev:0
  └─ Block('declarations')
    └─ VarDecl('i') → type:Integer tab:34 lev:0
    └─ VarDecl('sum') → type:Integer tab:35 lev:0
  └─ Block → btab:1 lev:1
    └─ Assign('sum' := 0) → type:Void
      └─ VarRef('sum') → type:Integer tab:35 lev:0
      └─ ConstInt('0') → type:Integer
    └─ Assign('i' := 1) → type:Void
      └─ VarRef('i') → type:Integer tab:34 lev:0
      └─ ConstInt('1') → type:Integer
    └─ WhileStmt → type:Void
      └─ BinOp('<=') → type:Boolean
        └─ VarRef('i') → type:Integer tab:34 lev:0
        └─ ConstInt('5') → type:Integer
      └─ Block
        └─ Assign('sum' := 'sum'+i') → type:Void
          └─ VarRef('sum') → type:Integer tab:35 lev:0
          └─ BinOp('+') → type:Integer
            └─ VarRef('sum') → type:Integer tab:35 lev:0
            └─ VarRef('i') → type:Integer tab:34 lev:0
        └─ Assign('i' := i'+1) → type:Void
          └─ VarRef('i') → type:Integer tab:34 lev:0
          └─ BinOp('+') → type:Integer
            └─ VarRef('i') → type:Integer tab:34 lev:0
            └─ ConstInt('1') → type:Integer
      └─ EmptyStmt

```

===== TAB (Identifier Table) =====

idx	name	obj	type	ref	nrm	lev	adr	link
0		variable	NoType	0	1	0	0	0
1	integer	type	Integer	0	1	0	0	0
2	real	type	Real	0	1	0	0	0
3	char	type	Char	0	1	0	0	0
4	boolean	type	Boolean	0	1	0	0	0
5	string	type	String	0	1	0	0	0
6	true	constant	Boolean	0	1	0	1	0
7	false	constant	Boolean	0	1	0	0	0
8	writeln	procedure	Void	0	1	0	0	0
9	readln	procedure	Void	0	1	0	0	0
10	write	procedure	Void	0	1	0	0	0

11	read	procedure	Void	0	1	0	0	0
33	TestWhile	program	Void	0	1	0	0	0
34	i	variable	Integer	0	1	0	0	33
35	sum	variable	Integer	0	1	0	1	34
===== BTAB (Block Table) =====								
idx	last	lpar	psze	vsze				

0	35	0	0	2				
1	0	0	0	0				
===== ATAB (Array Table) =====								
(kosong)								

Tabel 3.5 Input/Output Test Case 5

3.6 Test Case 6

Masukan (input)
<pre> program TestArray; var arr: array[1 .. 5] of integer; matrix: array[1 .. 3] of array[1 .. 3] of integer; i: integer; begin arr[1] := 10; arr[2] := 20; arr[3] := arr[2] + arr[1]; i := 3; arr[i] := arr[1] + arr[2]; writeln(arr[3]); matrix[1][1] := 99; matrix[2][3] := 42; writeln(matrix[1][1]); end. </pre>
Keluaran (output)
<pre> 10 20 30 99 </pre>

DECORATED ABSTRACT SYNTAX TREE

```

└─ Program('TestArray') → type:Void tab:33 btab:1 lev:0
  └─ Block('declarations')
    └─ VarDecl('arr') → type:Array tab:34 lev:0
    └─ VarDecl('matrix') → type:Array tab:35 lev:0
    └─ VarDecl('i') → type:Integer tab:36 lev:0
  └─ Block → btab:1 lev:1
    └─ Assign(ArrayAccess := 10) → type:Void
      └─ ArrayAccess → type:Integer
        └─ VarRef('arr') → type:Array tab:34 lev:0
        └─ ConstInt('1') → type:Integer
      └─ ConstInt('10') → type:Integer
    └─ Assign(ArrayAccess := 20) → type:Void
      └─ ArrayAccess → type:Integer
        └─ VarRef('arr') → type:Array tab:34 lev:0
        └─ ConstInt('2') → type:Integer
      └─ ConstInt('20') → type:Integer
    └─ Assign(ArrayAccess := ArrayAccess+ArrayAccess) → type:Void
      └─ ArrayAccess → type:Integer
        └─ VarRef('arr') → type:Array tab:34 lev:0
        └─ ConstInt('3') → type:Integer
      └─ BinOp('+') → type:Integer
        └─ ArrayAccess → type:Integer
          └─ VarRef('arr') → type:Array tab:34 lev:0
          └─ ConstInt('2') → type:Integer
        └─ ArrayAccess → type:Integer
          └─ VarRef('arr') → type:Array tab:34 lev:0
          └─ ConstInt('1') → type:Integer
    └─ Assign('i' := 3) → type:Void
      └─ VarRef('i') → type:Integer tab:36 lev:0
      └─ ConstInt('3') → type:Integer
    └─ Assign(ArrayAccess := ArrayAccess+ArrayAccess) → type:Void
      └─ ArrayAccess → type:Integer
        └─ VarRef('arr') → type:Array tab:34 lev:0
        └─ VarRef('i') → type:Integer tab:36 lev:0
      └─ BinOp('+') → type:Integer
        └─ ArrayAccess → type:Integer
          └─ VarRef('arr') → type:Array tab:34 lev:0
          └─ ConstInt('1') → type:Integer
        └─ ArrayAccess → type:Integer
          └─ VarRef('arr') → type:Array tab:34 lev:0
          └─ ConstInt('2') → type:Integer
    └─ writeln(...) → type:Void tab:8 lev:0

```

```

└─ ArrayAccess → type:Integer
    └─ VarRef('arr') → type:Array tab:34 lev:0
        └─ ConstInt('3') → type:Integer
└─ Assign(ArrayAccess := 99) → type:Void
    └─ ArrayAccess → type:Integer
        └─ ArrayAccess → type:Array
            └─ VarRef('matrix') → type:Array tab:35 lev:0
                └─ ConstInt('1') → type:Integer
            └─ ConstInt('1') → type:Integer
        └─ ConstInt('99') → type:Integer
└─ Assign(ArrayAccess := 42) → type:Void
    └─ ArrayAccess → type:Integer
        └─ ArrayAccess → type:Array
            └─ VarRef('matrix') → type:Array tab:35 lev:0
                └─ ConstInt('2') → type:Integer
            └─ ConstInt('3') → type:Integer
        └─ ConstInt('42') → type:Integer
└─ writeln(...) → type:Void tab:8 lev:0
    └─ ArrayAccess → type:Integer
        └─ ArrayAccess → type:Array
            └─ VarRef('matrix') → type:Array tab:35 lev:0
                └─ ConstInt('1') → type:Integer
            └─ ConstInt('1') → type:Integer
└─ EmptyStmt

```

===== TAB (Identifier Table) =====

idx	name	obj	type	ref	nrm	lev	adr	link
0		variable	NoType	0	1	0	0	0
1	integer	type	Integer	0	1	0	0	0
2	real	type	Real	0	1	0	0	0
3	char	type	Char	0	1	0	0	0
4	boolean	type	Boolean	0	1	0	0	0
5	string	type	String	0	1	0	0	0
6	true	constant	Boolean	0	1	0	1	0
7	false	constant	Boolean	0	1	0	0	0
8	writeln	procedure	Void	0	1	0	0	0
9	readln	procedure	Void	0	1	0	0	0
10	write	procedure	Void	0	1	0	0	0
11	read	procedure	Void	0	1	0	0	0
33	TestArray	program	Void	0	1	0	0	0
34	arr	variable	Array	0	1	0	0	33
35	matrix	variable	Array	1	1	0	5	34
36	i	variable	Integer	0	1	0	20	35

===== BTAB (Block Table) =====

idx	last	lpar	psize	vsze
0	36	0	0	21
1	0	0	0	0

0	36	0	0	21
1	0	0	0	0

0	36	0	0	21
1	0	0	0	0

===== ATAB (Array Table) =====

idx	xtyp	etyp	eref	low	high	elsz	size
0	Integer	Integer	0	1	5	1	5
1	Integer	Array	0	1	3	5	15

0	Integer	Integer	0	1	5	1	5
1	Integer	Array	0	1	3	5	15

0	Integer	Integer	0	1	5	1	5
1	Integer	Array	0	1	3	5	15

Tabel 3.6 Input/Output Test Case 6

3.7 Test Case 7

Masukan (input)

```

program TestRecord;
type
  Point == record
    x: integer;
    y: integer
  end;
var
  p: Point;
begin
  p.x := 3;
  p.y := 4;
  writeln(p.x);
  writeln(p.y);
end.

```

Keluaran (output)

DECORATED ABSTRACT SYNTAX TREE

└─ Program('TestRecord') → type:Void tab:33 btab:2 lev:0

```

└─ Block('declarations')
  └─ TypeDecl
    └─ TypeDecl('Point') → type:Record tab:36
      └─ TypeRecord
        └─ Block('fields')
          └─ FieldDecl
            └─ IdentList
              └─ VarRef('x') → type:Integer tab:34
                └─ TypeIdent('integer')
          └─ FieldDecl
            └─ IdentList
              └─ VarRef('y') → type:Integer tab:35
                └─ TypeIdent('integer')
    └─ VarDecl('p') → type:Record tab:37 lev:0
└─ Block → btab:2 lev:1
  └─ Assign(x := 3) → type:Void
    └─ FieldAccess('x') → type:Integer tab:34
      └─ VarRef('p') → type:Record tab:37 lev:0
    └─ ConstInt('3') → type:Integer
  └─ Assign(y := 4) → type:Void
    └─ FieldAccess('y') → type:Integer tab:35
      └─ VarRef('p') → type:Record tab:37 lev:0
    └─ ConstInt('4') → type:Integer
  └─ writeln(...) → type:Void tab:8 lev:0
    └─ FieldAccess('x') → type:Integer tab:34
      └─ VarRef('p') → type:Record tab:37 lev:0
  └─ writeln(...) → type:Void tab:8 lev:0
    └─ FieldAccess('y') → type:Integer tab:35
      └─ VarRef('p') → type:Record tab:37 lev:0
  └─ EmptyStmt

```

===== TAB (Identifier Table) =====

idx	name	obj	type	ref	nrm	lev	adr	link
0		variable	NoType	0	1	0	0	0
1	integer	type	Integer	0	1	0	0	0
2	real	type	Real	0	1	0	0	0
3	char	type	Char	0	1	0	0	0
4	boolean	type	Boolean	0	1	0	0	0
5	string	type	String	0	1	0	0	0
6	true	constant	Boolean	0	1	0	1	0
7	false	constant	Boolean	0	1	0	0	0
8	writeln	procedure	Void	0	1	0	0	0
9	readln	procedure	Void	0	1	0	0	0

10	write	procedure	Void	0	1	0	0	0
11	read	procedure	Void	0	1	0	0	0
33	TestRecord	program	Void	0	1	0	0	0
34	x	field	Integer	0	1	1	0	0
35	y	field	Integer	0	1	1	1	34
36	Point	type	Record	1	1	0	0	33
37	p	variable	Record	1	1	0	0	36

==== BTAB (Block Table) ====

idx	last	lpar	psze	vsze

0	37	0	0	2
1	35	0	0	2
2	0	0	0	0

==== ATAB (Array Table) ====

(kosong)

Tabel 3.7 Input/Output Test Case 7

3.8 Test Case 8

Masukan (input)
<pre> program TestProcedure; var result: integer; procedure add(a: integer; b: integer); begin result := a + b; end; begin result := 0; add(10, 20); writeln(result); end. </pre>
Keluaran (output)

DECORATED ABSTRACT SYNTAX TREE

```

└─ Program('TestProcedure') → type:Void tab:33 btab:2 lev:0
    └─ Block('declarations')
        └─ VarDecl('result') → type:Integer tab:34 lev:0
            └─ ProcDecl('add') → type:Void tab:35 btab:1 lev:1
                └─ IdentList('params')
                    └─ ParamDecl
                        └─ IdentList
                            └─ VarRef('a') → type:Integer tab:36 lev:1
                                └─ TypeIdent('integer')
                            └─ ParamDecl
                                └─ IdentList
                                    └─ VarRef('b') → type:Integer tab:37 lev:1
                                        └─ TypeIdent('integer')
                                └─ Block
                                    └─ Block
                                        └─ Assign('result' := 'a'+'b') → type:Void
                                            └─ VarRef('result') → type:Integer tab:34 lev:0
                                                └─ BinOp('+') → type:Integer
                                                    └─ VarRef('a') → type:Integer tab:36 lev:1
                                                        └─ VarRef('b') → type:Integer tab:37 lev:1
                                                    └─ EmptyStmt
                                                └─ EmptyStmt
                                            └─ EmptyStmt
                                        └─ EmptyStmt
                                    └─ EmptyStmt
                                └─ EmptyStmt
                            └─ EmptyStmt
                        └─ EmptyStmt
                    └─ EmptyStmt
                └─ EmptyStmt
            └─ EmptyStmt
        └─ EmptyStmt
    └─ Block → btab:2 lev:1
        └─ Assign('result' := 0) → type:Void
            └─ VarRef('result') → type:Integer tab:34 lev:0
                └─ ConstInt('0') → type:Integer
            └─ add(...) → type:Void tab:35 lev:0
                └─ ConstInt('10') → type:Integer
                └─ ConstInt('20') → type:Integer
            └─ writeln(...) → type:Void tab:8 lev:0
                └─ VarRef('result') → type:Integer tab:34 lev:0
            └─ EmptyStmt

```

===== TAB (Identifier Table) =====

idx	name	obj	type	ref	nrm	lev	adr	link
0		variable	NoType	0	1	0	0	0
1	integer	type	Integer	0	1	0	0	0
2	real	type	Real	0	1	0	0	0
3	char	type	Char	0	1	0	0	0
4	boolean	type	Boolean	0	1	0	0	0
5	string	type	String	0	1	0	0	0

6	true	constant	Boolean	0	1	0	1	0
7	false	constant	Boolean	0	1	0	0	0
8	writeln	procedure	Void	0	1	0	0	0
9	readln	procedure	Void	0	1	0	0	0
10	write	procedure	Void	0	1	0	0	0
11	read	procedure	Void	0	1	0	0	0
33	TestProcedure	program	Void	0	1	0	0	0
34	result	variable	Integer	0	1	0	0	33
35	add	procedure	Void	0	1	0	0	34
36	a	parameter	Integer	0	1	1	0	0
37	b	parameter	Integer	0	1	1	1	36

==== BTAB (Block Table) ====

idx	last	lpar	psize	vsze

0	35	0	0	1
1	37	37	2	0
2	0	0	0	0

==== ATAB (Array Table) ====

(kosong)

Tabel 3.8 Input/Output Test Case 8

3.9 Test Case 9

Masukan (input)
<pre> program TestFunction; var r: integer; function multiply(a: integer; b: integer): integer; begin multiply := a * b; end; begin r := multiply(6, 7); writeln(r); end. </pre>

Keluaran (output)

DECORATED ABSTRACT SYNTAX TREE

```

└─ Program('TestFunction') → type:Void tab:33 btab:2 lev:0
    └─ Block('declarations')
        └─ VarDecl('r') → type:Integer tab:34 lev:0
            └─ FuncDecl('multiply') → type:Integer tab:35 btab:1 lev:1
                └─ TypeIdent('integer')
                └─ IdentList('params')
                    └─ ParamDecl
                        └─ IdentList
                            └─ VarRef('a') → type:Integer tab:36 lev:1
                            └─ TypeIdent('integer')
                    └─ ParamDecl
                        └─ IdentList
                            └─ VarRef('b') → type:Integer tab:37 lev:1
                            └─ TypeIdent('integer')
                └─ Block
                    └─ Block
                        └─ Assign('multiply' := 'a'*'b') → type:Void
                            └─ VarRef('multiply') → type:Integer tab:35 lev:0
                            └─ BinOp('*') → type:Integer
                                └─ VarRef('a') → type:Integer tab:36 lev:1
                                └─ VarRef('b') → type:Integer tab:37 lev:1
                        └─ EmptyStmt
                    └─ Block
                        └─ Assign('r' := multiply(...)) → type:Void
                            └─ VarRef('r') → type:Integer tab:34 lev:0
                            └─ multiply(...) → type:Integer tab:35 lev:0
                                └─ ConstInt('6') → type:Integer
                                └─ ConstInt('7') → type:Integer
                        └─ writeln(...) → type:Void tab:8 lev:0
                            └─ VarRef('r') → type:Integer tab:34 lev:0
                        └─ EmptyStmt

```

===== TAB (Identifier Table) =====

idx	name	obj	type	ref	nrm	lev	adr	link
0		variable	NoType	0	1	0	0	0
1	integer	type	Integer	0	1	0	0	0
2	real	type	Real	0	1	0	0	0

3	char	type	Char	0	1	0	0	0
4	boolean	type	Boolean	0	1	0	0	0
5	string	type	String	0	1	0	0	0
6	true	constant	Boolean	0	1	0	1	0
7	false	constant	Boolean	0	1	0	0	0
8	writeln	procedure	Void	0	1	0	0	0
9	readln	procedure	Void	0	1	0	0	0
10	write	procedure	Void	0	1	0	0	0
11	read	procedure	Void	0	1	0	0	0
33	TestFunction	program	Void	0	1	0	0	0
34	r	variable	Integer	0	1	0	0	33
35	multiply	function	Integer	0	1	0	0	34
36	a	parameter	Integer	0	1	1	0	0
37	b	parameter	Integer	0	1	1	1	36

==== BTAB (Block Table) ====

idx	last	lpar	psze	vsze
0	35	0	0	1
1	37	37	2	0
2	0	0	0	0

==== ATAB (Array Table) ====

(kosong)

Tabel 3.9 Input/Output Test Case 9

3.10 Test Case 10

Masukan (input)
<pre> program TestScope; var globalVar: integer; procedure setGlobal(val: integer); begin globalVar := val; end; begin globalVar := 0; setGlobal(99); </pre>

```
writeln(globalVar);
end.
```

Keluaran (output)

DECORATED ABSTRACT SYNTAX TREE

```
└─ Program('TestScope') → type:Void tab:33 btab:2 lev:0
  └─ Block('declarations')
    └─ VarDecl('globalVar') → type:Integer tab:34 lev:0
    └─ ProcDecl('setGlobal') → type:Void tab:35 btab:1 lev:1
      └─ IdentList('params')
        └─ ParamDecl
          └─ IdentList
            └─ VarRef('val') → type:Integer tab:36 lev:1
            └─ TypeIdent('integer')
          └─ Block
            └─ Block
              └─ Assign('globalVar' := 'val') → type:Void
                └─ VarRef('globalVar') → type:Integer tab:34 lev:0
                └─ VarRef('val') → type:Integer tab:36 lev:1
              └─ EmptyStmt
            └─ Block → btab:2 lev:1
              └─ Assign('globalVar' := 0) → type:Void
                └─ VarRef('globalVar') → type:Integer tab:34 lev:0
                └─ ConstInt('0') → type:Integer
              └─ setGlobal(...) → type:Void tab:35 lev:0
                └─ ConstInt('99') → type:Integer
              └─ writeln(...) → type:Void tab:8 lev:0
                └─ VarRef('globalVar') → type:Integer tab:34 lev:0
              └─ EmptyStmt
```

===== TAB (Identifier Table) =====

idx	name	obj	type	ref	nrm	lev	adr	link
0		variable	NoType	0	1	0	0	0
1	integer	type	Integer	0	1	0	0	0
2	real	type	Real	0	1	0	0	0
3	char	type	Char	0	1	0	0	0
4	boolean	type	Boolean	0	1	0	0	0
5	string	type	String	0	1	0	0	0
6	true	constant	Boolean	0	1	0	1	0

7	false	constant	Boolean	0	1	0	0	0
8	writeln	procedure	Void	0	1	0	0	0
9	readln	procedure	Void	0	1	0	0	0
10	write	procedure	Void	0	1	0	0	0
11	read	procedure	Void	0	1	0	0	0
33	TestScope	program	Void	0	1	0	0	0
34	globalVar	variable	Integer	0	1	0	0	33
35	setGlobal	procedure	Void	0	1	0	0	34
36	val	parameter	Integer	0	1	1	0	0

==== BTAB (Block Table) ====

idx	last	lpar	psize	vsze

0	35	0	0	1
1	36	36	1	0
2	0	0	0	0

==== ATAB (Array Table) ====

(kosong)

Tabel 3.10 Input/Output Test Case 10

3.11 Test Case 11

Masukan (input)
<pre> program TestCase; var x, result: integer; begin x := 2; result := 0; case x of 1: result := 10; 2: result := 20; 3: result := 30; end; writeln(result); end. </pre>

Keluaran (output)

DECORATED ABSTRACT SYNTAX TREE

```

└─ Program('TestCase') → type:Void tab:33 btab:1 lev:0
  └─ Block('declarations')
    └─ VarDecl('x') → type:Integer tab:34 lev:0
    └─ VarDecl('result') → type:Integer tab:35 lev:0
  └─ Block → btab:1 lev:1
    └─ Assign('x' := 2) → type:Void
      └─ VarRef('x') → type:Integer tab:34 lev:0
      └─ ConstInt('2') → type:Integer
    └─ Assign('result' := 0) → type:Void
      └─ VarRef('result') → type:Integer tab:35 lev:0
      └─ ConstInt('0') → type:Integer
    └─ CaseStmt → type:Void
      └─ VarRef('x') → type:Integer tab:34 lev:0
      └─ CaseBlock
        └─ ConstInt('1') → type:Integer
        └─ Assign('result' := 10) → type:Void
          └─ VarRef('result') → type:Integer tab:35 lev:0
          └─ ConstInt('10') → type:Integer
        └─ CaseBlock
          └─ ConstInt('2') → type:Integer
          └─ Assign('result' := 20) → type:Void
            └─ VarRef('result') → type:Integer tab:35 lev:0
            └─ ConstInt('20') → type:Integer
          └─ CaseBlock
            └─ ConstInt('3') → type:Integer
            └─ Assign('result' := 30) → type:Void
              └─ VarRef('result') → type:Integer tab:35 lev:0
              └─ ConstInt('30') → type:Integer
        └─ writeln(...) → type:Void tab:8 lev:0
          └─ VarRef('result') → type:Integer tab:35 lev:0
      └─ EmptyStmt

```

===== TAB (Identifier Table) =====

idx	name	obj	type	ref	nrm	lev	adr	link
0		variable	NoType	0	1	0	0	0
1	integer	type	Integer	0	1	0	0	0
2	real	type	Real	0	1	0	0	0

3	char	type	Char	0	1	0	0	0
4	boolean	type	Boolean	0	1	0	0	0
5	string	type	String	0	1	0	0	0
6	true	constant	Boolean	0	1	0	1	0
7	false	constant	Boolean	0	1	0	0	0
8	writeln	procedure	Void	0	1	0	0	0
9	readln	procedure	Void	0	1	0	0	0
10	write	procedure	Void	0	1	0	0	0
11	read	procedure	Void	0	1	0	0	0
33	TestCase	program	Void	0	1	0	0	0
34	x	variable	Integer	0	1	0	0	33
35	result	variable	Integer	0	1	0	1	34

==== BTAB (Block Table) =====

idx	last	lpar	psze	vsze

0	35	0	0	2
1	0	0	0	0

==== ATAB (Array Table) =====

(kosong)

Tabel 3.11 Input/Output Test Case 11

3.12 Test Case 12

Masukan (input)
<pre> program TestTypes; type SmallInt == 1 .. 10; Color == (Red, Green, Blue); var n: SmallInt; c: Color; begin n := 5; writeln(n); end.</pre>

Keluaran (output)

DECORATED ABSTRACT SYNTAX TREE

```

└─ Program('TestTypes') → type:Void tab:33 btab:1 lev:0
    └─ Block('declarations')
        └─ TypeDecl
            └─ TypeDecl('SmallInt') → type:Subrange tab:34
                └─ TypeSubrange
                    └─ Range
                        └─ ConstInt('1')
                        └─ ConstInt('10')
            └─ TypeDecl('Color') → type:Enumerated tab:38
                └─ TypeEnumerated
                    └─ VarRef('Red') → type:Enumerated tab:35
                    └─ VarRef('Green') → type:Enumerated tab:36
                    └─ VarRef('Blue') → type:Enumerated tab:37
        └─ VarDecl('n') → type:Subrange tab:39 lev:0
        └─ VarDecl('c') → type:Enumerated tab:40 lev:0
    └─ Block → btab:1 lev:1
        └─ Assign('n' := 5) → type:Void
            └─ VarRef('n') → type:Subrange tab:39 lev:0
            └─ ConstInt('5') → type:Integer
        └─ writeln(...) → type:Void tab:8 lev:0
            └─ VarRef('n') → type:Subrange tab:39 lev:0
        └─ EmptyStmt

```

===== TAB (Identifier Table) =====

idx	name	obj	type	ref	nrm	lev	adr	link
0		variable	NoType	0	1	0	0	0
1	integer	type	Integer	0	1	0	0	0
2	real	type	Real	0	1	0	0	0
3	char	type	Char	0	1	0	0	0
4	boolean	type	Boolean	0	1	0	0	0
5	string	type	String	0	1	0	0	0
6	true	constant	Boolean	0	1	0	1	0
7	false	constant	Boolean	0	1	0	0	0
8	writeln	procedure	Void	0	1	0	0	0
9	readln	procedure	Void	0	1	0	0	0
10	write	procedure	Void	0	1	0	0	0
11	read	procedure	Void	0	1	0	0	0

33	TestTypes	program	Void	0	1	0	0	0
34	SmallInt	type	Subrange	0	1	0	0	33
35	Red	constant	Enumerated	0	1	0	0	34
36	Green	constant	Enumerated	0	1	0	1	35
37	Blue	constant	Enumerated	0	1	0	2	36
38	Color	type	Enumerated	0	1	0	0	37
39	n	variable	Subrange	0	1	0	0	38
40	c	variable	Enumerated	0	1	0	1	39

==== BTAB (Block Table) ====

idx	last	lpar	psze	vsze

0	40	0	0	2
1	0	0	0	0

==== ATAB (Array Table) ====

(kosong)

Tabel 3.12 Input/Output Test Case 12

3.13 Test Case 13

Masukan (input)
<pre> program TestGlobalMutate; var x: integer; procedure increment; begin x := x + 1; end; begin x := 0; increment; writeln(x); end. </pre>
Keluaran (output)
DECORATED ABSTRACT SYNTAX TREE

```

└─ Program('TestGlobalMutate') → type:Void tab:33 btab:2 lev:0
  └─ Block('declarations')
    └─ VarDecl('x') → type:Integer tab:34 lev:0
    └─ ProcDecl('increment') → type:Void tab:35 btab:1 lev:1
      └─ Block
        └─ Block
          └─ Assign('x' := 'x'+1) → type:Void
            └─ VarRef('x') → type:Integer tab:34 lev:0
            └─ BinOp('+') → type:Integer
              └─ VarRef('x') → type:Integer tab:34 lev:0
              └─ ConstInt('1') → type:Integer
          └─ EmptyStmt
        └─ Block → btab:2 lev:1
          └─ Assign('x' := 0) → type:Void
            └─ VarRef('x') → type:Integer tab:34 lev:0
            └─ ConstInt('0') → type:Integer
          └─ increment(...) → type:Void tab:35 lev:0
          └─ writeln(...) → type:Void tab:8 lev:0
            └─ VarRef('x') → type:Integer tab:34 lev:0
          └─ EmptyStmt

```

===== TAB (Identifier Table) =====

idx	name	obj	type	ref	nrm	lev	adr	link

0		variable	NoType	0	1	0	0	0
1	integer	type	Integer	0	1	0	0	0
2	real	type	Real	0	1	0	0	0
3	char	type	Char	0	1	0	0	0
4	boolean	type	Boolean	0	1	0	0	0
5	string	type	String	0	1	0	0	0
6	true	constant	Boolean	0	1	0	1	0
7	false	constant	Boolean	0	1	0	0	0
8	writeln	procedure	Void	0	1	0	0	0
9	readln	procedure	Void	0	1	0	0	0
10	write	procedure	Void	0	1	0	0	0
11	read	procedure	Void	0	1	0	0	0
33	TestGlobalMutateprogram		Void	0	1	0	0	0
34	x	variable	Integer	0	1	0	0	33
35	increment	procedure	Void	0	1	0	0	34

===== BTAB (Block Table) =====

```

idx  last  lpar  psze  vsze
-----

```

0	35	0	0	1
1	0	0	0	0
2	0	0	0	0

==== ATAB (Array Table) ====

(kosong)

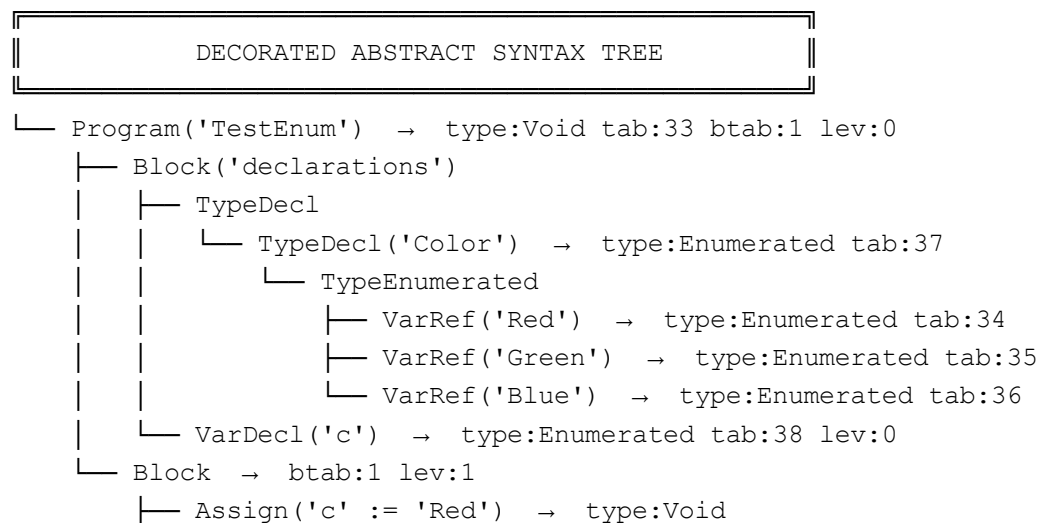
Tabel 3.13 Input/Output Test Case 13

3.14 Test Case 14

Masukan (input)

```
program TestEnum;
type
  Color == (Red, Green, Blue);
var
  c: Color;
begin
  c := Red;
  c := Green;
  c := Blue;
end.
```

Keluaran (output)



```

|   └─ VarRef('c') → type:Enumerated tab:38 lev:0
|   └─ VarRef('Red') → type:Enumerated tab:34 lev:0
└─ Assign('c' := 'Green') → type:Void
|   └─ VarRef('c') → type:Enumerated tab:38 lev:0
|   └─ VarRef('Green') → type:Enumerated tab:35 lev:0
└─ Assign('c' := 'Blue') → type:Void
|   └─ VarRef('c') → type:Enumerated tab:38 lev:0
|   └─ VarRef('Blue') → type:Enumerated tab:36 lev:0
└─ EmptyStmt

```

===== TAB (Identifier Table) =====

idx	name	obj	type	ref	nrm	lev	adr	link

0		variable	NoType	0	1	0	0	0
1	integer	type	Integer	0	1	0	0	0
2	real	type	Real	0	1	0	0	0
3	char	type	Char	0	1	0	0	0
4	boolean	type	Boolean	0	1	0	0	0
5	string	type	String	0	1	0	0	0
6	true	constant	Boolean	0	1	0	1	0
7	false	constant	Boolean	0	1	0	0	0
8	writeln	procedure	Void	0	1	0	0	0
9	readln	procedure	Void	0	1	0	0	0
10	write	procedure	Void	0	1	0	0	0
11	read	procedure	Void	0	1	0	0	0
33	TestEnum	program	Void	0	1	0	0	0
34	Red	constant	Enumerated	0	1	0	0	33
35	Green	constant	Enumerated	0	1	0	1	34
36	Blue	constant	Enumerated	0	1	0	2	35
37	Color	type	Enumerated	0	1	0	0	36
38	c	variable	Enumerated	0	1	0	0	37

===== BTAB (Block Table) =====

idx	last	lpar	psze	vsze

0	38	0	0	1
1	0	0	0	0

===== ATAB (Array Table) =====

(kosong)

Tabel 3.14 Input/Output Test Case 14

3.15 Test Case 15

Masukan (input)
<pre>program ErrUndeclared; var x: integer; begin x := 5; y := x + 1; end.</pre>
Keluaran (output)
[SEMANTIC ERROR] Identifier 'y' belum dideklarasikan.

Tabel 3.13 Input/Output Test Case 15

3.16 Test Case 16

Masukan (input)
<pre>program ErrRedeclare; var x: integer; x: real; begin x := 5; end.</pre>
Keluaran (output)
[SEMANTIC ERROR] Identifier 'x' sudah dideklarasikan dalam scope ini.

Tabel 3.16 Input/Output Test Case 16

3.17 Test Case 17

Masukan (input)

<pre> program ErrIfNotBool; var x: integer; begin x := 5; if x then begin x := 1; end; end. </pre>
Keluaran (output)
[SEMANTIC ERROR] Kondisi if-statement harus bertipe Boolean.

Tabel 3.17 Input/Output Test Case 17

3.18 Test Case 18

Masukan (input)
<pre> program ErrTypeMismatch; var x: integer; b: boolean; begin x := 5; b := true; x := b; end. </pre>
Keluaran (output)
[SEMANTIC ERROR] Assignment tidak kompatibel: target bertipe 'Integer', nilai bertipe 'Boolean'.

Tabel 3.18 Input/Output Test Case 18

3.19 Test Case 19

Masukan (input)
<pre> program ErrUninitialized; var result: integer; procedure compute; var local: integer; begin result := local + 1; end; begin compute; writeln(result); end. </pre>
Keluaran (output)
<pre> [SEMANTIC ERROR] Variabel tidak bisa dioperasikan jika belum diinisialisasi dengan nilai. </pre>

Tabel 3.19 Input/Output Test Case 19

3.20 Test Case 20

Masukan (input)
<pre> program ErrUndeclaredProc; var x: integer; begin x := 5; undeclaredProc(x); end. </pre>

Keluaran (output)
[SEMANTIC ERROR] Procedure/function 'undeclaredProc' belum dideklarasikan.

Tabel 3.20 Input/Output Test Case 20

3.21 Test Case 21

Masukan (input)
<pre> program ErrDivReal; var x: real; y: integer; begin x := 3.14; y := 5; y := y div x; end. </pre>
Keluaran (output)
[SEMANTIC ERROR] Operator 'div' tidak bisa diterapkan pada tipe 'Integer' dan 'Real'.

Tabel 3.21 Input/Output Test Case 21

3.22 Test Case 22

Masukan (input)
<pre> program ErrWhileNotBool; var x: integer; begin x := 5; while x do begin x := x - 1; </pre>

<pre> end; end. </pre>
Keluaran (output)
<pre> [SEMANTIC ERROR] Kondisi while-statement harus bertipe Boolean. </pre>

Tabel 3.22 Input/Output Test Case 22

3.23 Test Case 23

Masukan (input)
<pre> program ErrParamConflict; var x: integer; procedure setVal(x: integer); begin x := x + 1; end; begin x := 5; setVal(10); end. </pre>
Keluaran (output)
<pre> [SEMANTIC ERROR] Parameter 'x' konflik dengan identifier global yang sudah dideklarasikan. </pre>

Tabel 3.23 Input/Output Test Case 23

3.24 Test Case 24

Masukan (input)
<pre> program ErrEnumMismatch; type Color == (Red, Green, Blue); </pre>

<pre>var c: Color; x: integer; begin x := 5; c := x; end.</pre>
Keluaran (output)
<pre>[SEMANTIC ERROR] Assignment tidak kompatibel: target bertipe 'Enumerated', nilai bertipe 'Integer'.</pre>

Tabel 3.24 Input/Output Test Case 24

BAB IV

KESIMPULAN DAN SARAN

4.1 Kesimpulan

Pada Milestone 3, berhasil diimplementasikan beberapa tahap Semantic Analysis pada Arion Compiler. Dari pengerjaan ini dapat disimpulkan beberapa hal:

1. Semantic analysis terbukti dapat diimplementasikan secara modular dalam dua fase yang jelas terpisah, yaitu Syntax-Directed Translation untuk membentuk AST, dan dekorasi AST untuk mengisi symbol table serta melakukan type checking.
2. Tabel simbol (tab, btab, atab) dengan mekanisme linked list per blok dan display array terbukti efektif menangani seluruh konstruksi bahasa Arion, mencakup variabel global, parameter formal, variabel lokal, tipe array, tipe record, dan tipe enumerasi.
3. Sistem type checking yang diimplementasikan berhasil mendeteksi berbagai kategori error semantik, antara lain penggunaan identifier yang belum dideklarasikan, deklarasi dalam scope yang sama, ketidakcocokan tipe pada assignment, penggunaan tipe non-Boolean pada kondisi percabangan dan perulangan, serta penerapan operator aritmatika pada tipe yang tidak kompatibel.
4. Mekanisme lexical scoping dengan display array berhasil mendukung variable shadowing dan akses variabel global dari dalam prosedur bertingkat, sesuai dengan semantik bahasa Pascal yang menjadi acuan Arion.

4.2 Saran

Berdasarkan dari pengalaman mengerjakan Milestone 3, terdapat beberapa saran yang dapat menjadi pertimbangan untuk pengembangan, antara lain:

1. Pengecekan jumlah dan tipe argumen pada pemanggilan prosedur dan fungsi saat ini belum dilakukan secara ketat. Perlu ditambahkan pencocokan antara parameter aktual dan parameter formal berdasarkan urutan dan tipe.
2. Pengecekan inisialisasi variabel baru diimplementasikan sebagian. Perlu dikembangkan analisis alur data sederhana (*definite assignment analysis*) agar variabel yang belum tentu diinisialisasi di semua cabang kode dapat terdeteksi.

3. Dukungan tipe String yang lebih lengkap perlu ditambahkan, termasuk penanganan panjang string dan operasi konkatenasi dengan type checking yang tepat.
4. Optimasi performa lookup menggunakan hash table untuk symbol table utama, mengingat untuk program berskala besar pencarian linear pada linked list dapat menjadi bottleneck.

LAMPIRAN

Link Repository : <https://github.com/gabriellaalubis/OTH-Tubes-IF2224-2026>

Pembagian tugas :

Nama	NIM	Pembagian	Persentase
Gabriella Botimada Lubis	13524006	● Perancangan Semantic Rules dan Struktur AST ● Implementasi kod ● Pengujian Test Case ● Penyusunan Laporan	33.3%
Stefani Angeline Oroh	13524064	● Perancangan Semantic Rules dan Struktur AST ● Implementasi kod ● Pengujian Test Case ● Penyusunan Laporan	33.3%
Reva Natania Sitohang	13524098	● Perancangan Semantic Rules dan Struktur AST ● Implementasi kod ● Pengujian Test Case ● Penyusunan Laporan	33.3%

REFERENSI

- [1] GeeksforGeeks, “Semantic Analysis in Compiler Design.” [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/semantic-analysis-in-compiler-design/>. [Accessed: May 19, 2026].
- [2] GeeksforGeeks, “Symbol Table in Compiler.” [Online]. Available: <https://www.geeksforgeeks.org/symbol-table-compiler/>. [Accessed: May 19, 2026].
- [3] GeeksforGeeks, “Phases of a Compiler.” [Online]. Available: <https://www.geeksforgeeks.org/compiler-design/phases-of-a-compiler/>. [Accessed: May 19, 2026].
- [4] Stanford University, “CS143 Handout 14: Semantic Analysis.” [Online]. Available: <https://suif.stanford.edu/dragonbook/lecture-notes/Stanford-CS143/14-Semantic-Analysis.pdf>. [Accessed: May 19, 2026].